Thesis Progress Update

Eren Olmez
6 November 2026

1. Batch scaling and router training cost

This update follows the 29 October meeting: test the apparent batch-32 plateau, measure model residency and switching, reduce dispatch overhead, repeat router-energy measurements with warm-up, and test recovery after structural pruning. PC inference, Jetson estimates and the HPC classifier sweep are separate studies.

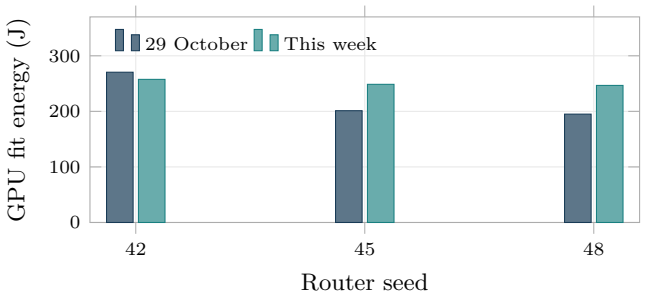
Does inference stop improving beyond batch 32? The historical routed curve flattened or worsened above 32. In the controlled follow-up, each comparison held the FP16 engine, model, validation inputs and input/output boundary fixed. The original TensorRT optimization profile was retained. With input already on the GPU and output left on-device, both tested models improved from batch 32 to 128:

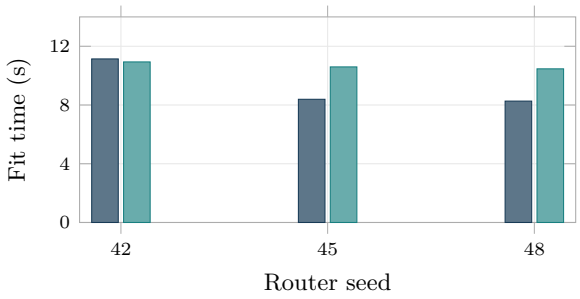
			GPU-resident input		Host input to host logits	
FP16 classifier	Batch	Test acc.	ms/image	GPU mJ/image	ms/image	GPU mJ/image
ResNet101	32	85.85%	0.3355	23.170	0.3632	25.509
	64	85.85%	0.3265	23.162	0.3518	24.313
	128	85.85%	0.3190	22.125	0.3457	23.512
EfficientNet-B0	32	82.71%	0.1384	9.895	0.1701	12.140
	64	82.71%	0.1242	8.862	0.1548	11.354
	128	82.71%	0.1164	8.046	0.1456	10.497

Top-1 accuracy was measured separately on all 10,000 CIFAR-100 test images at each batch size. Neither FP16 engine changed any predicted class across the three batch sizes. Timing and energy come from 36 validation-input windows (three repeats per setting). Against batch 32, GPU-resident batch 128 cut latency/energy by 4.92%/4.51% for ResNet101 and 15.91%/18.69% for EfficientNet-B0. No timed thermal slowdown was sampled; power-cap flags remained.

What caused the historical plateau? No single cause was isolated. The historical audit found zero cache evictions at batches 32–128, so repeated reloads cannot explain that part of the curve. A separate routed diagnostic observed thermal slowdown and power-cap flags. Its first-sweep latency includes initial engine loads; repeated-sweep GPU energy amortizes those loads. These observations identify plausible contributors, not a proven cause of the historical curve.

How does router fit cost compare with last week? The October 29 measurements and this week's warmed fits each cover 20 TinyCNN training epochs. This week's timed fit used less GPU energy for seed 42 but more for seeds 45 and 48. Fit time followed the same pattern.





Seed	29 Oct (s)	This week (s)	29 Oct (GPU J)	This week (GPU J)
42	11.138	10.934	270.437	257.57
45	8.389	10.595	201.185	248.65
48	8.266	10.463	195.008	246.69

One reported fit per seed on 29 October; medians of six fits per seed this week. This week's 100 dummy warm-up steps are outside the timed fit and cost energy themselves. These were separate sessions, so the differences do not isolate the effect of warm-up or replace last week's break-even estimates.

2. GPU residency and CPU–GPU dispatch

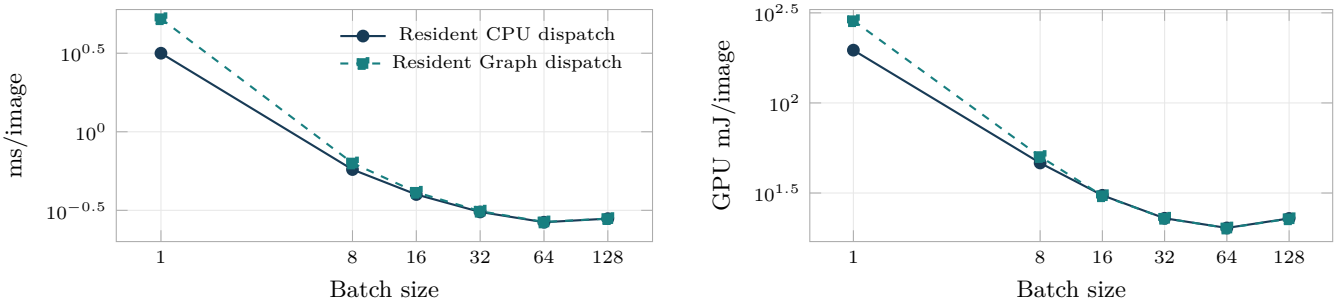
How many models can stay on the PC GPU? On the RTX 2060, all retained TensorRT engines re-executed with their own contexts and buffers while keeping at least 1 GiB of VRAM free. Engine order matters:

Batch	Smallest engines first	Largest engines first	Architecture round-robin
1	53	12	18
32	46	11	17
64	42	11	16
128	36	10	15

Demonstrated counts for these loading orders, not an absolute maximum or a test of holding all 213 configurations.

How often did the router switch models at batch 1? With the same six classifiers available, three separately trained and calibrated TinyCNN routers (seeds 42, 45, and 48) switched on 3,936 (39.4%), 5,010 (50.1%), and 5,207 (52.1%) of 9,999 consecutive image pairs, respectively. All saw the same 10,000 test images in different orders. The rates reflect both which models were chosen and the order of those choices, so they do not isolate the effect of training. A switch is a change in model choice, not a GPU reload.

What is a CUDA Graph? It records a sequence of GPU operations once so the CPU can submit the whole sequence for replay, instead of issuing the operations one by one. Here, after the CPU submits the replay, TinyCNN runs on the GPU and the GPU uses its scores to select and run just one of eight already-loaded classifiers. The control also runs inference on the GPU, but the CPU reads the routing scores and launches the selected classifier. Both paths made identical choices with no timed model reloads (seed 42).



Three-repeat medians on logarithmic axes. The eight-branch Graph worked correctly but did not make inference faster at any tested batch size. A separate two-model Graph saved 9.33% latency and 11.33% GPU energy; that result did not carry over to eight branches.

		Latency (ms/image)		GPU energy (mJ/image)		Graph cost change	
Batch	Val. acc.	CPU	Graph	CPU	Graph	Latency	Energy
1	85.16%	3.163	5.245	196.35	286.12	+65.83%	+45.72%
8	86.23%	0.575	0.633	46.43	50.40	+10.05%	+8.54%
16	86.04%	0.399	0.412	30.71	30.55	+3.31%	−0.54%
32	85.74%	0.309	0.312	22.90	22.90	+1.13%	+0.02%
64	85.06%	0.266	0.267	20.24	20.21	+0.39%	−0.12%
128	85.35%	0.280	0.280	22.85	22.75	+0.07%	−0.41%

Positive means more time or energy than CPU dispatch; percentages use unrounded medians. At batch 1, Graph took about 66% longer and used about 46% more GPU energy. At batches 16–128 the energy differences were below 1%. Predictions matched on all 1,024 calibration images.

Jetson Orin Nano 8 GB estimate. With a 3 GiB OS allowance and 1.5× PC allocation multiplier, the model predicts 24/3/9 engines for the same three order types, or about 11,001 FP32 128px tensors in a separate two-classifier-plus-router scenario. Models and images compete for shared memory. These are PC-based estimates, not measured Jetson capacity, latency or energy.

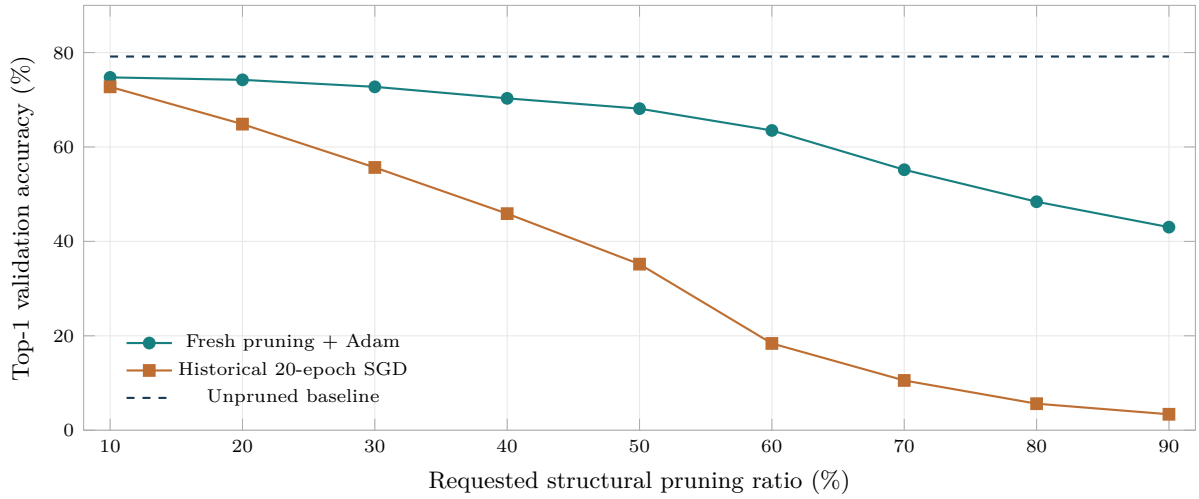
3. Fresh structural pruning and Adam fine-tuning

What did one epoch recover? Both Adam rates started from the same freshly pruned ResNet18 p30 weights. The unpruned 128px checkpoint scored 79.16% and the freshly pruned model 1.00% on the original 5,000-image validation split. Requested p30 removed 50.40% of parameters and 58.12% of MACs.

Adam learning rate	Epoch 1 validation	Epoch 5 validation	Best extended result
0.01	34.70%	58.38%	Not extended
0.001	58.10%	68.90%	72.74% (epoch 17)

At 0.001 the pilot stopped at epoch 22, five epochs after its last qualifying improvement. One epoch exceeded the historical p30 SGD validation reference (55.66%), but additional epochs recovered another 14.64 percentage points. This is not a matched Adam-versus-SGD optimizer ablation. On the official test set, this p30 checkpoint scored 71.82% overall: motorcycle, orchid, and sunflower tied for best (93%), and seal was worst (36%).

Does the recovery extend across pruning levels? All nine ResNet18 levels are complete in the new PC campaign. The plot shows the two sets of checkpoints on the original classifier validation split. The unpruned reference makes clear how much accuracy is left unrecovered at each pruning level.



Validation, not test accuracy. Requested pruning ratio is not the actual parameter or MAC reduction. The two recovery protocols differ; the separation is descriptive and not an isolated optimizer effect.

Requested ratio (%)	10	20	30	40	50	60	70	80	90
Adam val. (%)	74.74	74.22	72.74	70.30	68.12	63.50	55.18	48.40	43.02
SGD val. (%)	72.74	64.84	55.66	45.88	35.18	18.38	10.54	5.62	3.38

Fine-tuning is complete; benchmarking is separate. All 63 checkpoints were verified (27 trained on PC, 36 on HPC): seven architectures at nine pruning levels each. Each run started from the original trained weights before pruning and used Adam at 0.001. Checkpoints were selected on validation, not test. All 63 have been exported to ONNX for benchmarking; that does not yet establish a latency or energy result.

How many images repay router training energy?

Batch size	Images to break even
1	2,399
8	—
16	9,348
32	3,440
64	3,277
128	3,333

GPU energy only, without an accuracy filter. Baseline: always run unpruned ResNet101 FP32. The dash means no break-even point: at batch 8, routing used more energy per image. October 29 measurements; batch 1 used six resident models.

4. HPC training progress

HPC training from scratch. This sweep is separate from the fine-tuning bank. W&B recorded 359/1,728 completed runs (20.78%) and 72,276/345,600 epochs (20.91%). All four runs marked running had fresh metrics; 11 were marked failed or crashed, unchanged since the previous check. These counts do not verify Slurm state or checkpoints.

Active architecture (case)	Epoch	Latest val. acc.	Val. loss	Run-best val. acc.	Historical FP32 val.
ResNet50 (370)	101	75.04%	0.922	76.20%	83.94%
ResNet101 (371)	46	54.32%	1.747	58.22%	85.88%
MobileNetV3 Small (372)	62	27.92%	2.786	31.20%	61.82%
MobileNetV3 Large (373)	78	30.56%	2.698	37.16%	82.04%

Each run is planned for 200 epochs. Run-best is the highest validation accuracy observed so far in that run. The table below reports the highest observed validation accuracy across all runs of each architecture.

What is varied? Eight architectures, with 72 recipes each. Each recipe combines one optimizer with its allowed initial learning rate and weight decay, one learning-rate schedule, and CutMix on or off. All runs are planned for 200 epochs without early stopping.

Optimizer	Initial learning rates	Weight decay
SGD	0.03, 0.1, 0.2	0.0001, 0.0005
Adam	0.0003, 0.001, 0.003	0.0001, 0.0005
AdamW	0.0003, 0.001, 0.003	0.01, 0.05

Schedule: cosine or divide the rate by 10 at epochs 80 and 120. CutMix: off or $\alpha = 1$ (mixes patches and labels between training images). Each recipe is repeated to measure variability.

Architecture	Best validation (case)	Initial LR	Weight decay	Historical FP32 val.
ResNet18	80.12% (312)	0.1	0.0005	79.16%
ResNet34	81.66% (329)	0.1	0.0005	82.98%
ResNet50	81.26% (314)	0.1	0.0005	83.94%
ResNet101	81.82% (139)	0.03	0.0005	85.88%
MobileNetV3 Small	64.42% (228)	0.1	0.0001	61.82%
MobileNetV3 Large	72.52% (237)	0.1	0.0001	82.04%
EfficientNet-B0	72.70% (142)	0.03	0.0005	83.14%
ShuffleNetV2 ×1.0	75.48% (127)	0.03	0.0005	75.72%

Best observed validation is a single-run maximum, not a recipe ranking. All eight rows currently come from completed SGD runs with cosine scheduling and CutMix on. Current models train from random initialization at 32px. Historical unpruned TensorRT FP32 used ImageNet initialization, 128px inputs and a different validation split.

Next. Measure the new fine-tuned bank on PC at batches 1/8/16/32/64/128 with matched fixed-model and unpruned references. Use the same host-input-to-prediction boundary for each method and report ms/image and GPU mJ/image alongside the historical results. Continue auditing the separate HPC sweep.

5. Which classes are easiest and hardest after pruning?

We evaluated all 63 frozen fine-tuned checkpoints on the **official CIFAR-100 test set**: 10,000 images, 100 per class. Inference used PyTorch FP32 at 128px. The first table shows one checkpoint per architecture, chosen before evaluating the test set (10% requested pruning for all seven). Both heatmaps cover all nine pruning levels. Every accuracy on this page is test accuracy.

Architecture	Pruning	Overall	Worst class	Best class(es)
ResNet18	10%	75.07%	boy (41%)	skyscraper (97%)
ResNet34	10%	75.50%	shrew (40%)	motorcycle / mountain (+1 ties) (95%)
ResNet50	10%	76.02%	girl (36%)	motorcycle (97%)
ResNet101	10%	76.08%	seal (43%)	orange (96%)
MobileNetV3 Small	10%	60.32%	girl / otter (23%)	motorcycle / wardrobe (90%)
MobileNetV3 Large	10%	71.33%	woman (37%)	orange / road (95%)
EfficientNet-B0	10%	71.22%	seal (40%)	motorcycle (96%)

How does the weakest class change as pruning increases?

Requested pruning (%)	10	20	30	40	50	60	70	80	90
ResNet18	41	33	36	38	30	17	13	3	1
ResNet34	40	45	28	43	25	29	10	5	4
ResNet50	36	39	42	40	45	24	29	6	3
ResNet101	43	39	37	38	30	16	6	3	2
MobileNetV3 Small	23	18	21	24	22	16	17	8	4
MobileNetV3 Large	37	32	27	23	29	24	26	13	6
EfficientNet-B0	40	18	23	35	26	17	24	15	8

Each cell is the lowest class accuracy (%) at that pruning level; darker means higher accuracy. The weakest class can change from cell to cell.

How does the strongest class change as pruning increases?

Requested pruning (%)	10	20	30	40	50	60	70	80	90
ResNet18	97	95	93	96	94	91	91	86	82
ResNet34	95	97	96	94	95	96	92	89	88
ResNet50	97	97	96	96	96	95	95	90	89
ResNet101	96	95	98	96	98	96	87	86	78
MobileNetV3 Small	90	90	89	92	89	86	87	87	82
MobileNetV3 Large	95	94	93	91	94	94	89	89	90
EfficientNet-B0	96	96	94	94	92	93	89	94	89

Each cell is the highest class accuracy (%) at that pruning level, using the same color scale as above. The strongest class can change from cell to cell.

Where do models disagree most on class accuracy? At each selected pruning level, this is the class with the largest accuracy gap between models.

Pruning	Class	Highest accuracy	Lowest accuracy
10%	man	ResNet101 (76%)	MobileNetV3 Small (31%)
50%	leopard	ResNet50 (82%)	MobileNetV3 Small (44%)
90%	orange	ResNet50 (89%)	ResNet34 (42%)

Test results were not used to train or choose checkpoints. Differences in class accuracy suggest where models might complement one another, but do not establish a routing gain.